INFORME DE EVIDENCIAS TÉCNICAS

Frontend E-commerce en Next.js 16

Optimización avanzada de rendimiento: Web Vitals y mutaciones asíncronas en el servidor

Actividad	Actividad evaluada individual · Kodigo Full Stack Jr 36 · Módulo 4
Autor	Hugo Jovel
Fecha	15 de octubre de 2026
Stack	Next.js 16.4.0 (App Router) · TypeScript · Tailwind CSS v4 · Server Components y Server Actions
API consumida	API REST de e-commerce (Laravel 12 · JWT · Swagger · Stripe), Módulo 3
Entorno	Debian XFCE 13 · VS Code · Node.js · MariaDB
Alcance	10 capturas de pantalla documentadas, hallazgos de integración y evidencias pendientes

1. Resumen ejecutivo

Este informe documenta, figura por figura, las capturas obtenidas durante la construcción y prueba del frontend de e-commerce en Next.js 16 que consume la API REST de Laravel. Cada evidencia se analiza con el mismo esquema: **qué muestra**, **qué requisito respalda**, **detalle técnico** y **observaciones o hallazgos**.

La arquitectura mantiene el token JWT exclusivamente en el servidor (cookie `httpOnly`), realiza las lecturas con Server Components, las mutaciones con Server Actions y evita interfaces desactualizadas con `revalidateTag` / `revalidatePath`. Las pruebas de integración contra la API real detectaron **dos diferencias de contrato** (dirección de envío obligatoria y endpoint de pago distinto), documentadas en la sección 4 con su plan de corrección.

Estado de cumplimiento frente a la actividad

Requisito	Estado y evidencia
Catálogo público (listado y detalle)	Parcial: los productos del carrito provienen del catálogo (Fig. 2). Falta captura del listado y del detalle.
Registro y login con la API	Evidenciado: registro (Fig. 1) y sesión activa (Fig. 2).
Token en cookie httpOnly	Pendiente: capturar la cookie en DevTools → Application → Cookies.
Carrito en estado local	Evidenciado (Fig. 2).
Crear orden mediante mutación	En ajuste: la API exige `shipping_address` (Fig. 4, H-01).
Pago con Stripe y confirmación	En ajuste: el contrato real es `/payments/checkout` (Fig. 8, H-02). Falta captura de confirmación.
Historial de compras protegido	Pendiente de captura (ruta implementada, Fig. 10).
loading.tsx, error.tsx y Suspense	Evidenciado: skeletons por Suspense (Fig. 6) y manejo de errores (Fig. 4).
Endpoints desde Swagger	Evidenciado (Fig. 7, 8 y 9). Falta el cuerpo de solicitud de POST /orders y /payments/checkout.
Reporte de rendimiento Lighthouse	Pendiente: ejecutar `npm run lighthouse` sobre la build de producción.
Lint y build sin errores	Evidenciado (Fig. 10).

2. Contexto y arquitectura

El navegador nunca habla directamente con la API: toda llamada pasa por el servidor de Next.js, que adjunta el Bearer leído de la cookie `httpOnly`. Esto elimina la exposición del token a scripts del cliente y de la URL de la API (`BASE_URL` es solo de servidor).

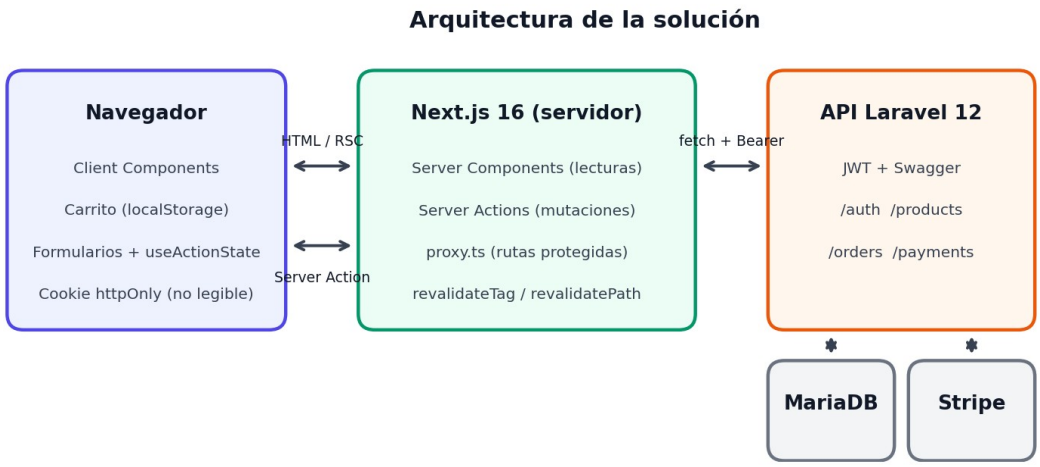


Diagrama 1. Flujo entre navegador, servidor Next.js, API Laravel, MariaDB y Stripe.

Decisiones de diseño

- **Seguridad:** JWT en cookie `httpOnly` + `sameSite=lax` (+ `secure` en producción); `proxy.ts` como primera barrera y verificación en el componente como defensa en profundidad; parámetro `next` validado contra open-redirect.
- **Integridad:** el navegador envía solo `product_id` y `quantity`; precios y totales los calcula la API.
- **Rendimiento:** Server Components, streaming con Suspense, skeletons con dimensiones fijas, JavaScript de cliente mínimo, fuentes del sistema y caché de datos públicos (`revalidate: 60`, tag `products`).
- **Resiliencia:** `error.tsx` por segmento, errores de validación mostrados en línea y reintento de pago sin duplicar la orden.

3. Evidencias y análisis por figura

Índice de figuras:

Figura	Contenido
Fig. 1	Registro de usuario (/registro)
Fig. 2	Carrito con sesión activa (/carrito)
Fig. 3	Checkout protegido (/checkout)
Fig. 4	Validación devuelta por la API en el checkout
Fig. 5	API REST en línea (GET /)
Fig. 6	Build de producción y streaming con Suspense
Fig. 7	Swagger: Autenticación y Productos
Fig. 8	Swagger: Órdenes, Pagos y esquemas
Fig. 9	Swagger: esquemas de datos
Fig. 10	Calidad de entrega: lint y build

Figura 1. Registro de usuario (/registro)

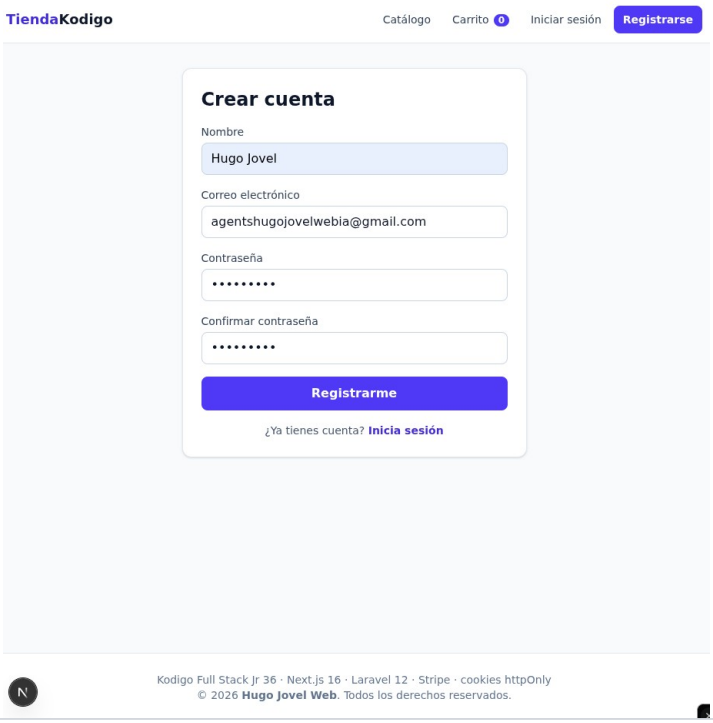


Figura 1. Formulario de registro de TiendaKodigo, ruta /registro, con el header en estado invitado.

Qué muestra	Formulario con Nombre, Correo electrónico, Contraseña y Confirmar contraseña. El header muestra “Iniciar sesión / Registrarse” y “Carrito 0” (usuario sin sesión). El footer incluye el sello de autoría.
Requisito que respalda	Autenticación: registro consumiendo la API.
Detalle técnico	Client Component con `useActionState` que invoca la Server Action `registerAction`. La validación se repite en servidor (nombre ≥ 2, correo válido, contraseña ≥ 8, confirmación idéntica) antes de llamar `POST /auth/register`. Los errores por campo regresan en el estado del formulario y se enlazan con `aria-describedby`. Si la API no devuelve token al registrar, la acción inicia sesión automáticamente y guarda el JWT en una cookie `httpOnly`.
Observaciones	Los campos de contraseña no se vuelven a pintar tras un error (`defaultValue` vacío). La captura contiene un correo real: conviene difuminarlo si el informe se publica en un repositorio público.

Figura 2. Carrito con sesión activa (/carrito)

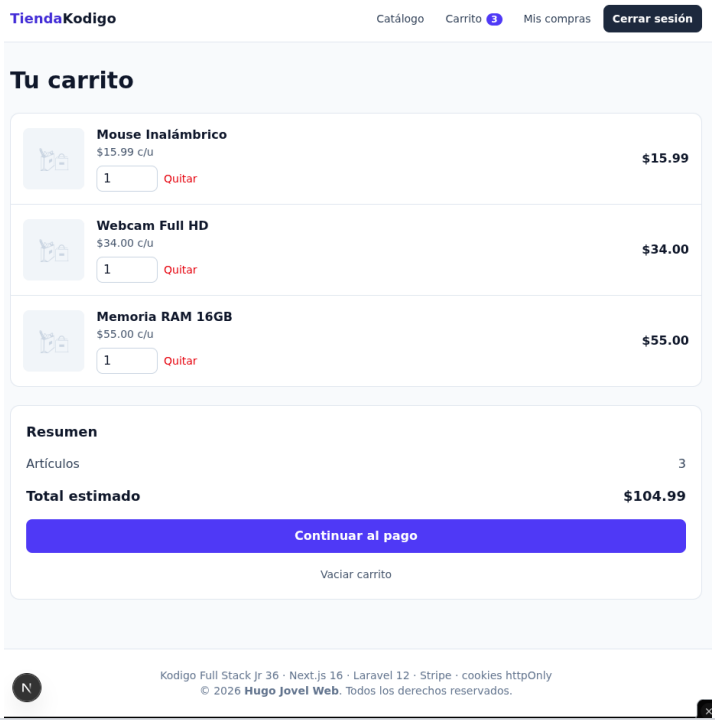


Figura 2. Carrito con tres productos y total estimado de $104.99; header con sesión iniciada.

Qué muestra	Tres líneas: Mouse Inalámbrico ($15.99), Webcam Full HD ($34.00) y Memoria RAM 16GB ($55.00), con cantidad editable y acción “Quitar”. Resumen: 3 artículos, total estimado $104.99. El header ya muestra “Mis compras” y “Cerrar sesión” y el contador del carrito en 3.
Requisito que respalda	Carrito y orden: agregar o quitar productos (estado local). Autenticación: sesión activa tras el registro.
Detalle técnico	`cart-store.ts` implementa el carrito con `useSyncExternalStore` sobre `localStorage` (clave `ecom_cart_v1`), sincronizado entre pestañas con el evento `storage` y con `getServerSnapshot` constante para evitar desajustes de hidratación. Solo se guardan id, nombre, precio de visualización, imagen y cantidad (limitada a 1–99). El token no se almacena aquí.
Observaciones	Los importes son informativos (“Total estimado”): la API recalcula el total a partir de `product_id` y `quantity`. Las miniaturas son un marcador porque los productos de ejemplo no traen imagen; `ProductImage` reserva dimensiones fijas, sin saltos de diseño.

Figura 3. Checkout protegido (/checkout)

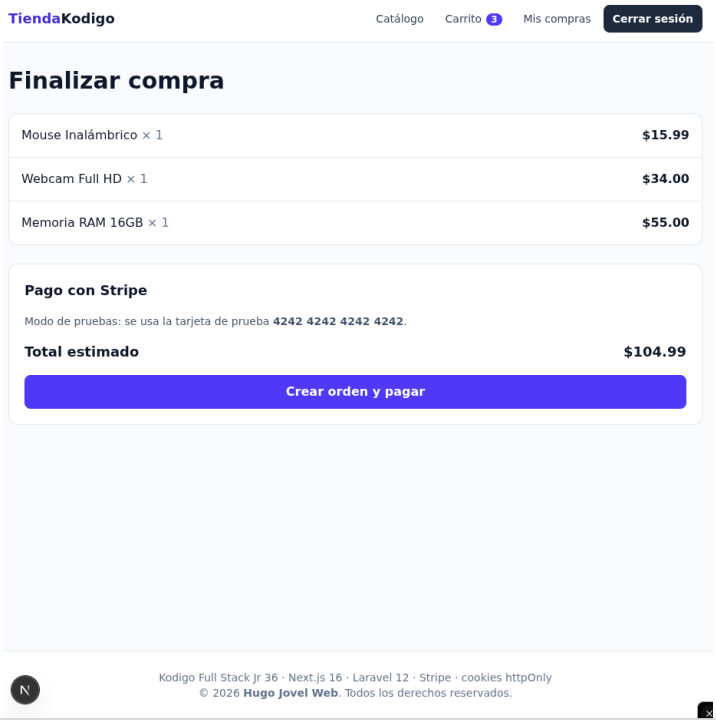


Figura 3. Pantalla “Finalizar compra” previa al envío del pago.

Qué muestra	Resumen de las tres líneas, panel “Pago con Stripe” con la tarjeta de prueba 4242 4242 4242 4242, total estimado de $104.99 y el botón “Crear orden y pagar”.
Requisito que respalda	Carrito y orden: crear orden mediante mutación. Pago con Stripe. Vista protegida.
Detalle técnico	La ruta se protege en dos capas: `proxy.ts` (Next.js 16) redirige a `/login?next=` si falta la cookie y el Server Component vuelve a verificar con `getToken()`. El formulario envía solo `{ product_id, quantity }`; `checkoutAction` (Server Action) crea la orden y ejecuta el pago en el servidor usando el Bearer leído de la cookie httpOnly. `SubmitButton` usa `useFormStatus` para mostrar el estado pendiente y evitar doble envío.
Observaciones	Es el estado previo al envío. La respuesta de la API a este formulario se documenta en la Figura 4.

Figura 4. Validación devuelta por la API en el checkout

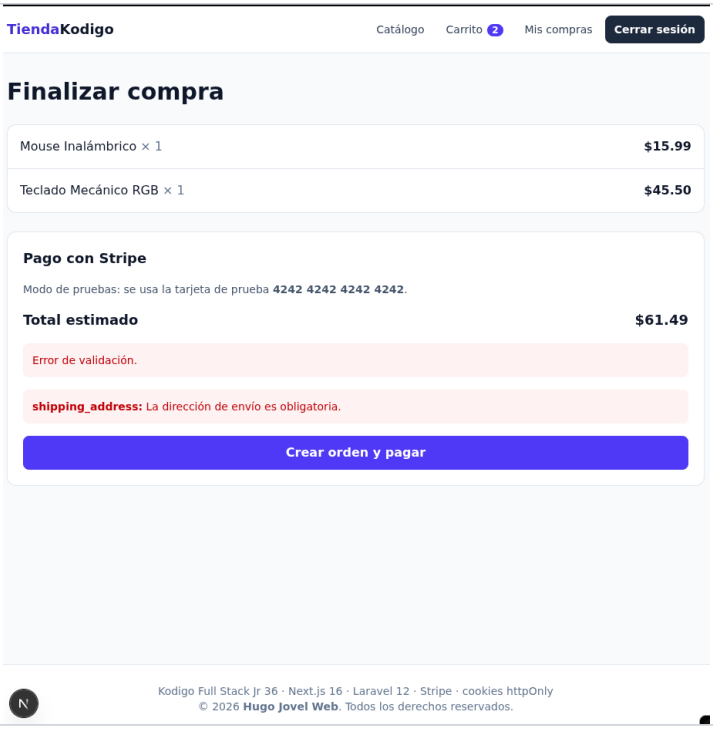


Figura 4. Error de validación del servidor mostrado en el formulario de checkout (carrito de 2 productos, $61.49).

Qué muestra	Tras pulsar “Crear orden y pagar” aparece “Error de validación.” y, debajo, “shipping_address: La dirección de envío es obligatoria.”. Carrito: Mouse Inalámbrico ($15.99) y Teclado Mecánico RGB ($45.50).
Requisito que respalda	Requisitos de resiliencia: control de errores en mutaciones asíncronas.
Detalle técnico	`apiFetch` lanza un `ApiError` con `status`, `message` y `errors` (cuerpo 422 de Laravel). `checkoutAction` lo convierte en `FormState { message, errors }` sin romper la página: un error de validación es esperado, por lo que se muestra en línea y no se delega a `error.tsx`. `CheckoutForm` lista los errores por campo con `role="alert"`.
Observaciones	**Hallazgo H-01.** La API exige `shipping_address` en `POST /orders` (ver Figura 8, esquema “Orden de compra”) y el formulario aún no captura ese dato. La prueba detectó una brecha de contrato; la corrección se describe en la sección 4.

Figura 5. API REST en línea (GET /)



Figura 5. Respuesta JSON de http://127.0.0.1:8000/ (verificación de salud de la API).

Qué muestra	Respuesta `{ "app": "Ecommerce API", "status": "ok", "docs": "http://127.0.0.1:8000/api/documentation" }`.
Requisito que respalda	Prerrequisito del caso de estudio: API REST de e-commerce (Laravel 12) operativa.
Detalle técnico	Servida por `php artisan serve` en el puerto 8000. El frontend la consume a través de `BASE_URL=http://localhost:8000/api`, variable solo de servidor (sin prefijo `NEXT_PUBLIC_`), por lo que la URL de la API no se expone al navegador.
Observaciones	Es el primer chequeo ante el mensaje “No se pudo conectar con la API” (503) que genera `ApiError` cuando `fetch` falla. Si en otro equipo `localhost` resolviera a IPv6, usar `127.0.0.1` en `BASE_URL`.

Figura 6. Build de producción y streaming con Suspense

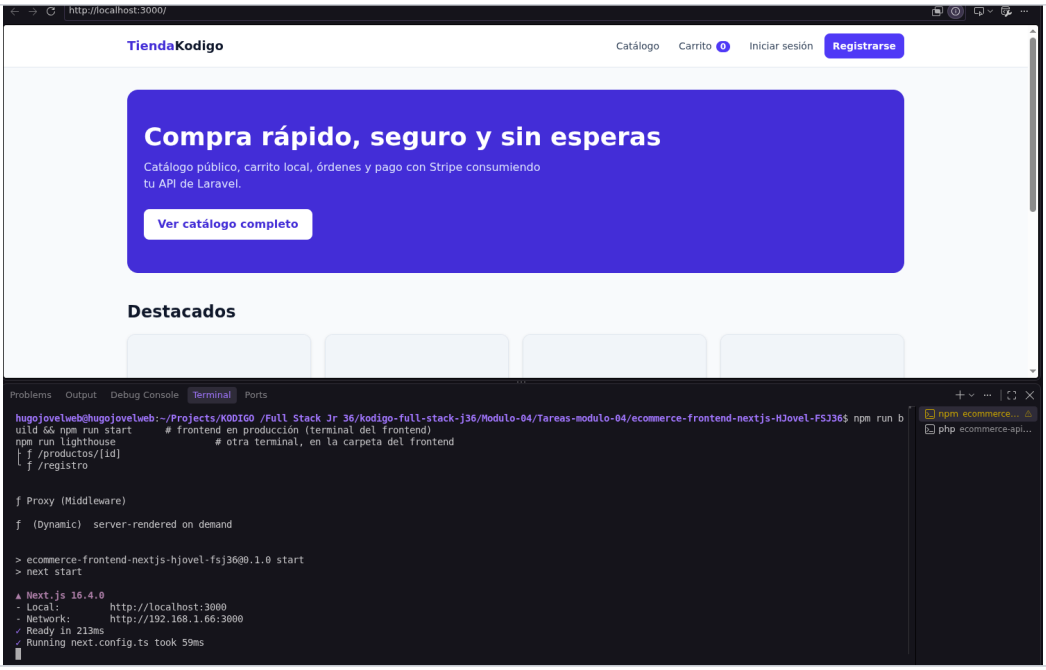


Figura 6. Inicio con next start y portada mostrando los skeletons de “Destacados”.

Qué muestra	Portada con el hero y la sección “Destacados” con tarjetas grises (skeletons). En el terminal: salida de `next build` y arranque de Next.js 16.4.0 en http://localhost:3000, “Ready in 213ms”.
Requisito que respalda	Rendimiento: `loading.tsx` / Suspense y medición sobre build de producción (Lighthouse).
Detalle técnico	Las tarjetas grises son el fallback de `<Suspense>` alrededor de `ProductGrid`, un Server Component asíncrono: el HTML estructural se envía por streaming antes de que responda la API. La lista de rutas de la build marca *ƒ* (dinámicas, renderizadas bajo demanda) y `Proxy (Middleware)`. La medición se hace con `next start`, no con `next dev`.
Observaciones	Los skeletons reservan el mismo espacio que el contenido final (relación 4:3) para minimizar el CLS. La captura se tomó en pleno streaming; se recomienda adjuntar también el catálogo ya cargado.

Figura 7. Swagger: Autenticación y Productos

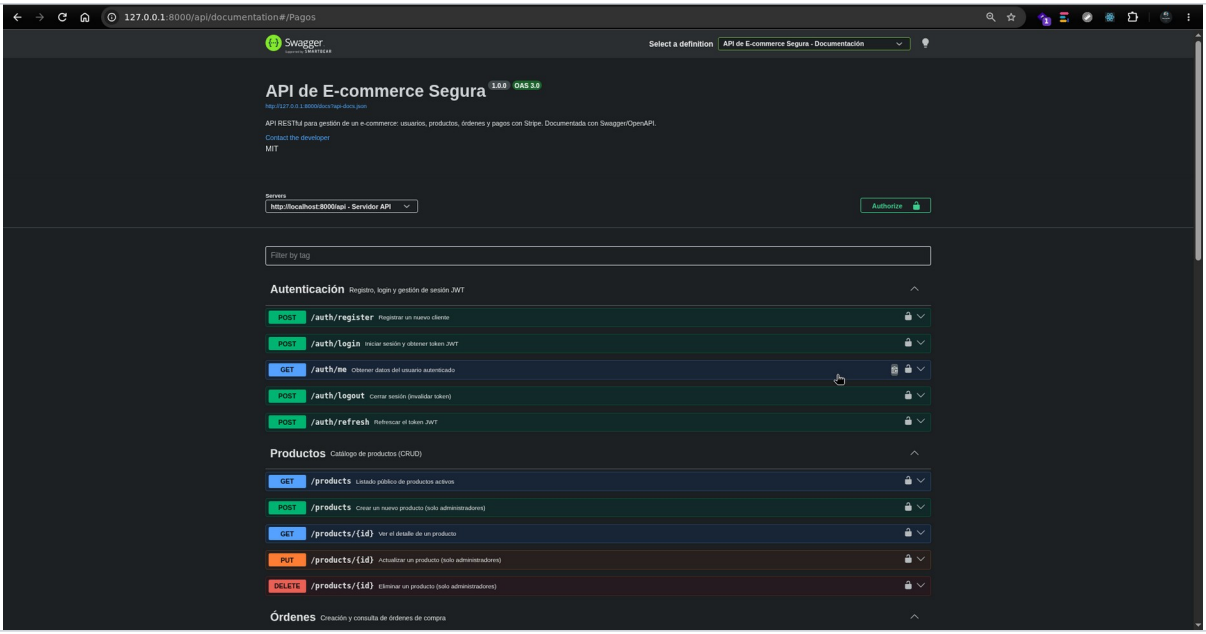


Figura 7. Swagger UI (http://127.0.0.1:8000/api/documentation) con los grupos Autenticación y Productos.

Qué muestra	"API de E-commerce Segura" v1.0.0 (OAS 3.0), servidor http://localhost:8000/api y botón Authorize (JWT). Autenticación: POST /auth/register, POST /auth/login, GET /auth/me, POST /auth/logout, POST /auth/refresh. Productos: GET y POST /products, GET, PUT y DELETE /products/{id}.
Requisito que respalda	Entregable: endpoints consumidos desde Swagger.
Detalle técnico	El frontend consume `register`, `login`, `logout`, `GET /products` y `GET /products/{id}`. Las lecturas públicas usan `fetch` con `next: { revalidate: 60, tags: ["products"] }`. `/auth/me` y `/auth/refresh` están disponibles pero no se usan en esta versión. La escritura de productos queda restringida a administradores.
Observaciones	`/auth/refresh` permitiría renovar el JWT antes de su expiración; se anota como mejora futura (H-05).

Figura 8. Swagger: Órdenes, Pagos y esquemas

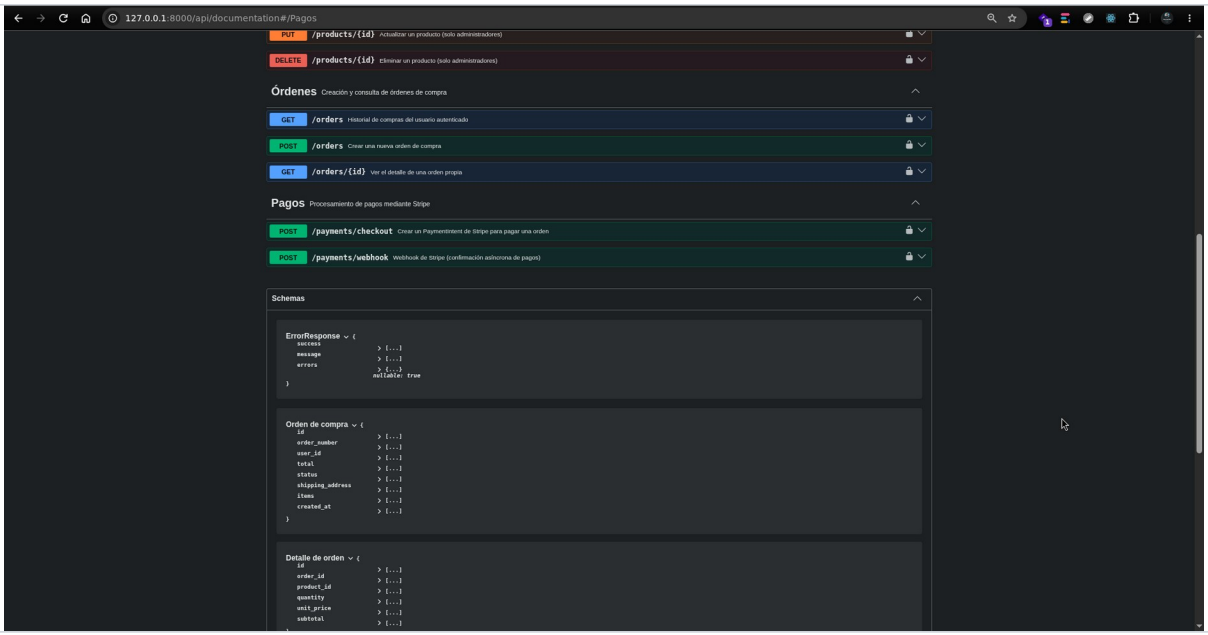


Figura 8. Grupos Órdenes y Pagos, y primeros esquemas (ErrorResponse y Orden de compra).

Qué muestra	Órdenes: GET /orders (historial del usuario autenticado), POST /orders (crear orden), GET /orders/{id}. Pagos: POST /payments/checkout (crea un PaymentIntent de Stripe para pagar una orden) y POST /payments/webhook (confirmación asíncrona). Esquemas: ErrorResponse (success, message, errors) y Orden de compra (id, order_number, user_id, total, status, shipping_address, items, created_at).
Requisito que respalda	Entregable: endpoints consumidos desde Swagger. Flujo de pago con Stripe e historial.
Detalle técnico	Historial = `GET /orders`; confirmación = `GET /orders/{id}`. El pago se modela con PaymentIntent: `/payments/checkout` recibe la orden y devuelve los datos del intento, y la confirmación definitiva llega por webhook (asíncrona), por lo que el estado de la orden puede tardar en pasar a pagado.
Observaciones	**Hallazgo H-02.** El frontend asumió `POST /orders/{id}/pay`; el contrato real es `POST /payments/checkout`. Además, `shipping_address` forma parte de la orden (confirma H-01). Tras pagar conviene mostrar un estado “procesando” y revalidar el historial.

Figura 9. Swagger: esquemas de datos

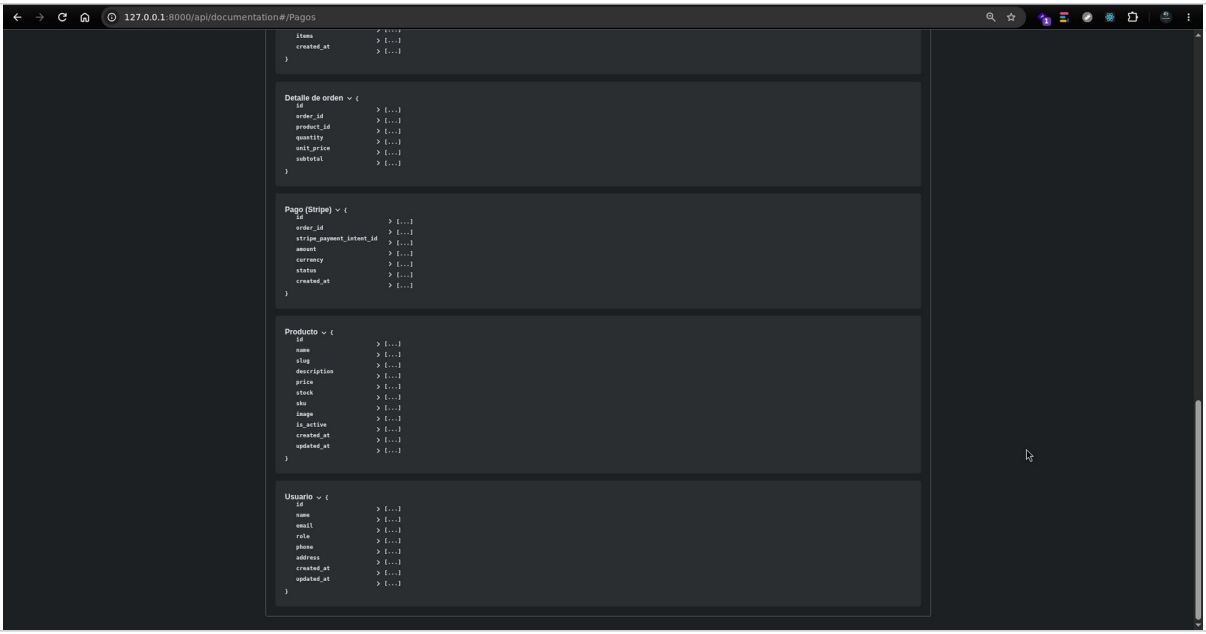


Figura 9. Esquemas Detalle de orden, Pago (Stripe), Producto y Usuario.

Qué muestra	Detalle de orden (id, order_id, product_id, quantity, unit_price, subtotal), Pago (Stripe) (id, order_id, stripe_payment_intent_id, amount, currency, status, created_at), Producto (id, name, slug, description, price, stock, sku, image, is_active, created_at, updated_at) y Usuario (id, name, email, role, phone, address, created_at, updated_at).
Requisito que respalda	Contrato de datos tipado que respalda los normalizadores del frontend.
Detalle técnico	`data.ts` mapea `Producto.image` → `Product.image`, `price` (decimal) → número, y `Detalle de orden.unit_price` → `OrderItem.price`. `ErrorResponse { success, message, errors }` es compatible con la extracción de `message` y `errors` de `ApiError`. El esquema de Pago incluye `stripe_payment_intent_id` y `status`.
Observaciones	`order_number` podría mostrarse en el historial en lugar del id interno. `is_active` indica que el catálogo público solo lista productos activos.

Figura 10. Calidad de entrega: lint y build

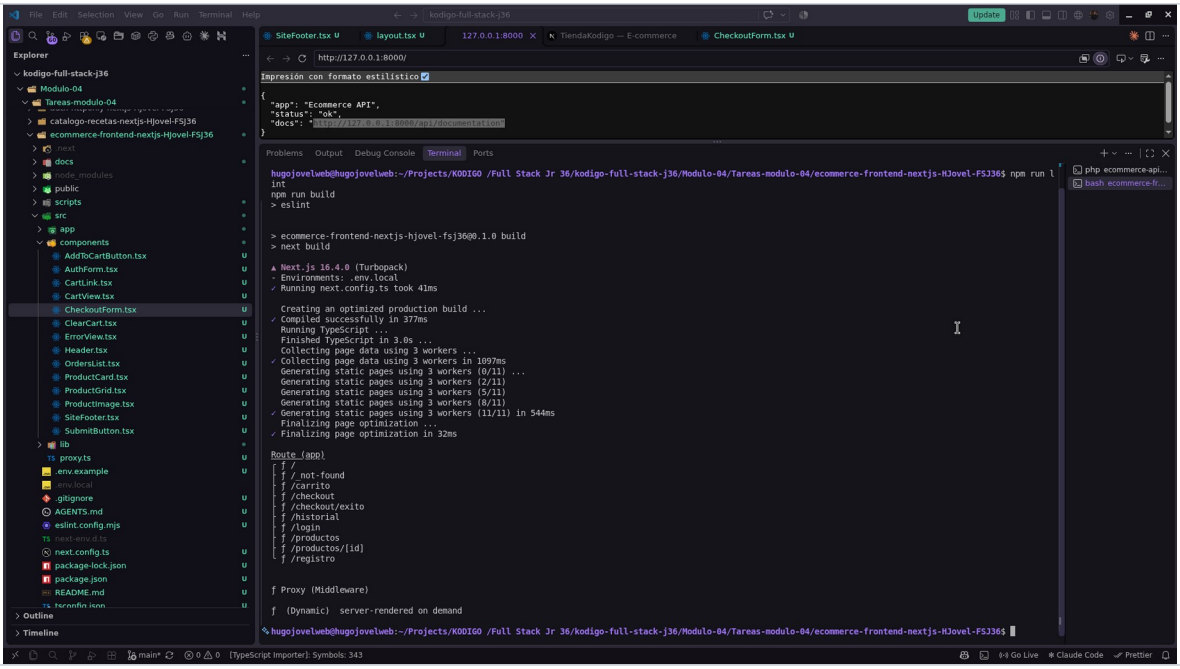


Figura 10. Resultado de npm run lint y npm run build sin errores.

Qué muestra	ESLint sin hallazgos y build de Next.js 16.4.0 (Turbopack) con `.env.local`: compilación en 377 ms, TypeScript en 3.0 s y 11 páginas generadas. Rutas: /, /_not-found, /carrito, /checkout, /checkout/exito, /historial, /login, /productos, /productos/[id], /registro, más `Proxy (Middleware)`. El explorador muestra `src/components` con `SiteFooter.tsx`.
Requisito que respalda	Calidad de la entrega y rutas implementadas (catálogo, autenticación, checkout, historial).
Detalle técnico	Todas las rutas son dinámicas (*ƒ*) porque el Header lee la cookie de sesión; la caché de datos se conserva en las lecturas públicas con `revalidate: 60` y tags. Las rutas protegidas (`/checkout`, `/historial`) se cubren con `proxy.ts` y verificación en el propio componente.
Observaciones	Sin errores de ESLint ni de tipos. Es la condición previa para publicar el repositorio.

4. Hallazgos de integración y plan de corrección

Las pruebas contra la API real revelaron diferencias entre el contrato asumido al diseñar el frontend y el contrato publicado en Swagger. Se registran con su causa, acción y estado.

ID	Hallazgo	Acción	Estado
H-01	`POST /orders` rechaza la orden: `shipping_address` obligatorio (Fig. 4 y 8).	Agregar el campo “Dirección de envío” al formulario de checkout (obligatorio, con `trim` y longitud acotada) y enviarlo en el cuerpo de la orden; mostrar errores por campo.	Pendiente
H-02	El pago real es `POST /payments/checkout` (PaymentIntent), no `/orders/{id}/pay` (Fig. 8).	Actualizar `ENDPOINTS` y `checkoutAction` para llamar al endpoint real con el id de la orden; tratar la confirmación como asíncrona (webhook) y revalidar el historial.	Pendiente
H-03	Error `SQLSTATE[HY000] [1045] Access denied` en la API al consultar productos.	Recrear el usuario de MariaDB con la contraseña del `.env` de la API, conceder privilegios sobre la base y ejecutar `php artisan config:clear`.	Resuelto
H-04	Productos de ejemplo sin imagen: el frontend muestra un marcador (Fig. 2).	Cargar el campo `image` en el seeder o aceptar el marcador; no afecta a la funcionalidad.	Opcional
H-05	`/auth/refresh` disponible y no utilizado: el JWT expira sin renovación.	Hoy un 401 en checkout redirige a `/login`. Mejora futura: renovar el token con `/auth/refresh`.	Mejora futura

5. Evidencias pendientes para la entrega

Para completar el paquete de evidencias solicitado en la actividad faltan estas capturas:

- Listado del catálogo con productos cargados y vista de detalle de un producto.
- Cookie `ecom_token` con la marca **HttpOnly** (DevTools → Application → Cookies).
- Cuerpo de solicitud y respuesta de `POST /orders` y `POST /payments/checkout` en Swagger.
- Flujo completo tras corregir H-01 y H-02: checkout, pantalla de confirmación y orden en `/historial`.
- Pantalla de `error.tsx` (con la API apagada) y de `loading.tsx` / skeletons con throttling de red.
- Reporte Lighthouse de `/productos`, `/productos/1` y `/carrito` sobre la build de producción, con una breve lectura de LCP, CLS y TBT.

6. Conclusiones

La base técnica está completa y verificada: build y lint sin errores, rutas protegidas, carrito local, API operativa y documentada en Swagger, y un manejo de errores que traslada las validaciones del servidor a la interfaz. Las pruebas de integración cumplieron su propósito al detectar dos diferencias de contrato (H-01 y H-02) antes de la entrega. Corregidas esas dos piezas y capturadas las evidencias pendientes (sección 5), el paquete cubre todos los entregables solicitados.

Anexo A. Endpoints de la API y su uso en el frontend

Método	Endpoint	Uso	Auth	Frontend
POST	/auth/register	Registro	No	Sí
POST	/auth/login	Inicio de sesión (JWT)	No	Sí
POST	/auth/logout	Cierre de sesión	Bearer	Sí
GET	/auth/me · POST /auth/refresh	Perfil y renovación de token	Bearer	No (mejora futura)
GET	/products	Catálogo público paginado	No	Sí
GET	/products/{id}	Detalle de producto	No	Sí
POST	/orders	Crear orden	Bearer	Sí (ajuste H-01)
GET	/orders	Historial del usuario	Bearer	Sí
GET	/orders/{id}	Confirmación / detalle	Bearer	Sí
POST	/payments/checkout	Crear PaymentIntent de Stripe	Bearer	Pendiente (H-02)
POST	/payments/webhook	Confirmación asíncrona de Stripe	Stripe	No (lo usa Stripe)

Anexo B. Rutas implementadas

Ruta	Tipo	Acceso	Notas
/	Servidor	Público	Hero y destacados con Suspense
/productos	Servidor	Público	Catálogo paginado · loading.tsx · error.tsx · Suspense
/productos/[id]	Servidor	Público	Detalle · notFound() · metadata dinámica
/login · /registro	Servidor + Server Action	Invitados	Validación y errores de la API
/carrito	Cliente (estado local)	Público	Agregar, quitar y cambiar cantidad
/checkout	Servidor + Server Action	Protegido	Crear orden y pagar · loading.tsx · error.tsx
/checkout/exito	Servidor	Protegido	Confirmación de compra
/historial	Servidor	Protegido	Órdenes del usuario · Suspense · loading.tsx · error.tsx